

Guia de Estudos: Estrutura de Dados III

Instituto Federal Sul-rio-grandense - Câmpus Passo Fundo

Bacharelado em Ciência da Computação

Disciplina: Estrutura de Dados III

Professor: Prof. Leonardo Deliyannis Constantin

Tópico: Counting Sort e Radix Sort (LSD & MSD)

1. Introdução e Limites da Ordenação

Na maioria dos algoritmos clássicos estudados (Quick Sort, Merge Sort, Heap Sort, Insertion Sort), a ordenação é executada por meio de comparações sequenciais entre pares de chaves ($A[i] \leq A[j]$).

- **O Limite Teórico por Comparação:** Formalmente, qualquer algoritmo baseado em comparação pode ser modelado como uma **Árvore de Decisão Binária**:
 - Cada nó interno representa uma comparação entre dois elementos ($A[i] \leq A[j]$).
 - Cada folha representa uma das possíveis permutações das chaves de entrada.
 - Para um vetor de tamanho n , existem $n!$ permutações possíveis.
 - Para distinguir entre $n!$ folhas, uma árvore binária precisa de altura $\lceil \log_2(n!) \rceil$.
 - Pela Aproximação de Stirling ($\ln(n!) \approx n \ln n - n$), temos que $\log_2(n!) = \Omega(n \log n)$.
 - Portanto, nenhum algoritmo baseado em comparação consegue ter melhor desempenho no pior caso do que $\Omega(n \log n)$.
- **Quebrando a Barreira (Ordenação em Tempo Linear):** Algoritmos como o Counting Sort e o Radix Sort não realizam comparações diretas de grandeza

entre os itens. Eles utilizam a própria representação e propriedades matemáticas das chaves (valores inteiros discretos, representação posicional em bases numéricas ou strings) como índices diretos de acesso em memória, possibilitando tempo de execução linear $O(n)$ sob condições favoráveis de

2. Counting Sort (Ordenação por Contagem)

O Counting Sort assume que cada elemento de entrada pertence a um conjunto discreto de inteiros no intervalo $[0, k]$.

Etapas do Algoritmo

1. **Contagem de Ocorrências:** Cria-se um vetor auxiliar C com tamanho $k + 1$, inicializado com zeros. Para cada chave presente no vetor original A , incrementa-se $C[A[j]]$.
2. **Somas Acumuladas (Prefixadas):** Calcula-se a soma cumulativa em C :
$$3. \ C[i] = C[i] + C[i - 1], \forall i \in [1, k]$$
4. Agora, $C[i]$ armazena quantos elementos no conjunto são menores ou iguais a i . Essa informação determina o índice exato onde a chave i deve ser posicionada no vetor final.
5. **Distribuição Estável no Vetor de Saída:** Percorre-se o vetor A obrigatoriamente da direita para a esquerda (do índice n até 1):
 - O elemento $A[j]$ é alocado na posição $C[A[j]]$ do vetor de saída B .
 - O contador $C[A[j]]$ é decrementado em 1.
 - *Nota pedagógica:* Essa varredura de trás para frente é o que garante a estabilidade do algoritmo (elementos iguais mantêm a ordem original).

Análise de Complexidade e Propriedades

- **Tempo no Melhor Caso:** $\Theta(n + k)$
- **Tempo no Médio Caso:** $\Theta(n + k)$
- **Tempo no Pior Caso:** $\Theta(n + k)$
- **Espaço Auxiliar:** $\Theta(n + k)$ (necessita do vetor auxiliar C de tamanho $k + 1$ e do vetor de saída B de tamanho n).
- **In-Place?** Não.
- **Estável?** Sim.

- **Quando usar?** Quando $k = O(n)$ (o valor máximo das chaves não cresce muito mais rápido que a quantidade de elementos).
- **Quando evitar?** Quando $k \gg n$ (ex: ordenar 10 inteiros cujos valores podem chegar a 1.000.000, gerando desperdício massivo de memória e processamento inútil no vetor C).

```
#include <stdio.h>
#include <stdlib.h>
```

```
void countingSort(int arr[], int n) {
    if (n <= 1) return;

    // 1. Encontrar o valor mínimo e o máximo para determinar a amplitude k
    int min = arr[0];
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < min) min = arr[i];
        if (arr[i] > max) max = arr[i];
    }

    int k = max - min + 1; // Faixa de valores possíveis

    // 2. Alocação dos vetores auxiliares
    int* count = (int*)calloc(k, sizeof(int));
    int* output = (int*)malloc(n * sizeof(int));

    if (!count || !output) {
        fprintf(stderr, "Erro de alocação de memória.\n");
        exit(EXIT_FAILURE);
    }

    // 3. Contagem das frequências simples (com deslocamento para tratar negativos)
    for (int i = 0; i < n; i++) {
        count[arr[i] - min]++;
    }
}
```

```
}
```

```
// 4. Somas cumulativas / prefixadas ( $C[i] = C[i] + C[i-1]$ )
```

```
for (int i = 1; i < k; i++) {  
    count[i] += count[i - 1];  
}
```

```
// 5. Distribuição no vetor de saída (Varredura reversa para garantir  
estabilidade)
```

```
for (int i = n - 1; i >= 0; i--) {  
    int val = arr[i];  
    int pos = count[val - min] - 1;  
    output[pos] = val;  
    count[val - min]--;  
}
```

```
// 6. Cópia do resultado de volta para o vetor original
```

```
for (int i = 0; i < n; i++) {  
    arr[i] = output[i];  
}
```

```
// Liberação de memória
```

```
free(count);  
free(output);  
}
```

```
int main() {
```

```
    int arr[] = {4, -2, 2, 8, 3, 3, 1, -2};  
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
    printf("Counting Sort - Entrada:\n");  
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);  
    printf("\n");
```

```
    countingSort(arr, n);
```

```

printf("Counting Sort - Ordenado:\n");
for (int i = 0; i < n; i++) printf("%d ", arr[i]);
printf("\n\n");

return 0;
}

```

3. Radix Sort: LSD vs. MSD

O Radix Sort soluciona o problema de chaves com valores altos dividindo os números em d dígitos em uma determinada base b .

3.1 Radix Sort LSD (Least Significant Digit)

- **Princípio:** Inicia a ordenação pelo dígito menos significativo (unidade) e avança progressivamente para a esquerda (dezenas, centenas, etc.).
- **Regra Mandatória:** A sub-rotina de ordenação usada para ordenar cada dígito **PRECISA SER ESTÁVEL**. Se a ordenação do dígito não for estável, a ordenação dos dígitos inferiores anteriores é desfeita.
- **Complexidade:**
 - **Tempo:** $\Theta(d \cdot (n + b))$ para melhor, pior e médio caso.
 - **Espaço:** $\Theta(n + b)$.
- **Vantagem:** Muito simples de implementar iterativamente, ideal para números inteiros ou chaves de tamanho fixo.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Retorna o valor máximo do vetor
```

```
int getMax(int arr[], int n) {
```

```
int mx = arr[0];  
  
for (int i = 1; i < n; i++) {  
    if (arr[i] > mx) mx = arr[i];  
}  
  
return mx;  
}
```

// Counting Sort estável acionado por cada posição pos (1 = unidades, 10 = dezenas, ...)

```
void countingSortByDigit(int arr[], int n, int exp) {  
  
    int output[n];  
  
    int count[10] = {0}; // Base 10 (dígitos de 0 a 9)  
  
    // Contagem de ocorrências do dígito atual  
    for (int i = 0; i < n; i++) {  
        int digit = (arr[i] / exp) % 10;  
        count[digit]++;  
    }  
  
    // Somas prefixadas  
    for (int i = 1; i < 10; i++) {  
        count[i] += count[i - 1];  
    }  
}
```

```
// Construção estável da saída (varredura reversa mandatória)
```

```
for (int i = n - 1; i >= 0; i--) {  
    int digit = (arr[i] / exp) % 10;  
    output[count[digit] - 1] = arr[i];  
    count[digit]--;  
}
```

```
// Copia os dados ordenados por este dígito de volta ao vetor original
```

```
for (int i = 0; i < n; i++) {  
    arr[i] = output[i];  
}  
}
```

```
// Algoritmo Radix Sort LSD principal
```

```
void radixSortLSD(int arr[], int n) {
```

```
    if (n <= 1) return;
```

```
    int m = getMax(arr, n);
```

```
    // Itera exp = 1, 10, 100, ... enquanto houver dígitos no maior número
```

```
    for (int exp = 1; m / exp > 0; exp *= 10) {
```

```

        countingSortByDigit(arr, n, exp);
    }
}

int main() {
    int arr[] = {170, 45, 75, 90, 802, 24, 2, 66};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Radix Sort LSD - Entrada:\n");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");

    radixSortLSD(arr, n);

    printf("Radix Sort LSD - Ordenado:\n");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n\n");

    return 0;
}

```

3.2 Radix Sort MSD (Most Significant Digit)

- **Princípio:** Inicia a ordenação pelo dígito mais significativo e avança para a direita utilizando partição recursiva (Divisão e Conquista).
- **Funcionamento:** Agrupa os elementos em baldes (*buckets*) baseados no primeiro dígito. Em seguida, chama o algoritmo recursivamente para cada balde considerando o próximo dígito.
- **Complexidade:**
 - **Tempo de Melhor Caso:** $\Theta(n)$ (quando os elementos já se diferenciam fortemente nos primeiros dígitos).
 - **Pior Caso:** $\Theta(d \cdot (n + b))$.
 - **Espaço:** $\Theta(n + d \cdot b)$ (devido à pilha de chamadas de recursão e criação de baldes).
- **Vantagem:** Excelente para ordenar strings e sequências de comprimentos variáveis.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define R 256 // Tamanho do alfabeto (ASCII estendido)
```

```
// Função auxiliar que retorna o caractere no índice d ou -1 se a string encerrou
```

```
int charAt(const char* s, int d) {
```

```
    if (d < (int)strlen(s)) {
```

```
        return (unsigned char)s[d];
```

```
    }
```

```
    return -1; // Marcador de fim de string
```

```
}
```

```

// Rotina recursiva do Radix Sort MSD

void radixSortMSD_rec(char* a[], int lo, int hi, int d, char* aux[]) {

    if (hi <= lo) return;

    // count[0..R+1]:

    // count[0] para strings que terminaram (charAt == -1)

    // count[1..R] para cada caractere ASCII

    int count[R + 2] = {0};

    // 1. Contagem de frequências

    for (int i = lo; i <= hi; i++) {

        int c = charAt(a[i], d);

        count[c + 2]++;

    }

    // 2. Somas cumulativas para delimitar as fronteiras dos baldes

    for (int r = 0; r < R + 1; r++) {

        count[r + 1] += count[r];

    }

    // 3. Distribuição dos elementos no vetor auxiliar

    for (int i = lo; i <= hi; i++) {

```

```
int c = charAt(a[i], d);  
  
aux[count[c + 1]++] = a[i];  
  
}
```

// 4. Copia os elementos de volta para o vetor original no intervalo [lo, hi]

```
for (int i = lo; i <= hi; i++) {  
  
    a[i] = aux[i - lo];  
  
}
```

// 5. Chamadas recursivas para cada balde de caracteres (ignora o balde r = 0 que são as strings finalizadas)

```
for (int r = 0; r < R; r++) {  
  
    radixSortMSD_rec(a, lo + count[r], lo + count[r + 1] - 1, d + 1, aux);  
  
}
```

// Wrapper principal do Radix Sort MSD

```
void radixSortMSD(char* a[], int n) {  
  
    char** aux = (char**)malloc(n * sizeof(char*));  
  
    if (!aux) {  
  
        fprintf(stderr, "Erro de memória.\n");  
  
        exit(EXIT_FAILURE);  
  
    }
```

```
    radixSortMSD_rec(a, 0, n - 1, 0, aux);

    free(aux);
}
```

```
int main() {

    char* palavras[] = {

        "algoritmo",

        "arvore",

        "radix",

        "counting",

        "analise",

        "avl",

        "ordenacao",

        "alocacao"

    };

    int n = sizeof(palavras) / sizeof(palavras[0]);


    printf("Radix Sort MSD - Entrada:\n");

    for (int i = 0; i < n; i++) printf("%s\n", palavras[i]);

    printf("\n");


    radixSortMSD(palavras, n);
}
```

```

printf("Radix Sort MSD - Ordenado Lexicograficamente:\n");

for (int i = 0; i < n; i++) printf("%s\n", palavras[i]);

printf("\n");

return 0;
}

```

4. Tabela Comparativa

Algoritmo	Complexidade de Tempo	Espaço Auxiliar	Técnica Chave
Counting Sort	$\theta(n + k)$	$\theta(n + k)$	Indexação direta por histograma
Radix Sort LSD	$\theta(d \cdot (n + b))$	$\theta(n + b)$	Passadas estáveis dígito a dígito
Radix Sort MSD	$\theta(n)$ a $\theta(d \cdot (n + b))$	$\theta(n + d \cdot b)$	Particionamento recursivo por baldes

Analogias Visuais:

- **Counting Sort:** Uma estante com escaninhos numerados fixos onde jogamos fichas numeradas e depois recolhemos em ordem sequencial.

- **Radix Sort LSD:** Classificação manual de cartões perfurados do censo: passa-se a agulha na coluna das unidades, depois das dezenas, mantendo os cartões agrupados.
- **Radix Sort MSD:** Organização de um fichário telefônico em ordem alfabética: separa-se tudo pela primeira letra 'A', 'B', 'C'..., e depois organiza-se internamente cada gaveta pela segunda letra.

5. Questões

Questão 01: O Limite Assintótico e a Ruptura de Paradigma (POSCOMP)

Enunciado:

No modelo de árvore de decisão para algoritmos de ordenação, qual é o limite inferior assintótico no pior caso para qualquer algoritmo baseado estritamente na comparação de chaves, e por qual razão o Counting Sort e o Radix Sort conseguem superar essa barreira?

- **A)** $\Omega(n)$; superam porque operam de forma estritamente *in-place* sem consumo de memória adicional.
- **B)** $\Omega(\log n)$; superam através da escolha ideal de pivôs que dividem o problema em subproblemas de tamanho $n/2$.
- **C)** $\Omega(n \log n)$; superam porque não realizam comparações diretas de ordem entre pares de chaves ($A[i] < A[j]$), utilizando o valor numérico dos elementos como índice direto de acesso à memória.
- **D)** $\Omega(n^2)$; superam porque exigem que a entrada esteja quase-ordenada antes da execução.
- **E)** $\Omega(n \log n)$; superam através da construção de uma árvore binária de busca balanceada em tempo constante.

Análise das Alternativas Incorretas:

- **Por que a A está incorreta?** Primeiro, $\Omega(n)$ é o limite do melhor caso de alguns algoritmos de comparação (como Insertion Sort), nunca o limite inferior teórico do pior caso geral. Segundo, o Counting Sort e o Radix Sort não são *in-place*; eles exigem memória auxiliar $\Theta(n + k)$ para funcionar.
- **Por que a B está incorreta?** $\Omega(\log n)$ é a complexidade típica de busca binária ou altura de árvores balanceadas, não de ordenação completa de n itens. Além disso, divisão com pivô ideal é a estratégia do Quick Sort, que é baseado em comparação e limitado a $\Omega(n \log n)$.

- **Por que a D está incorreta?** $\Omega(n^2)$ é o limite de algoritmos ineficientes como Bubble Sort e Selection Sort em determinados cenários, não a barreira teórica mínima universal de comparação. Além disso, Counting e Radix não dependem de pré-ordenação dos dados.
- **Por que a E está incorreta?** Construir uma árvore binária balanceada (como AVL ou Rubro-Negra) a partir de n elementos custa $n \times O(\log n) = O(n \log n)$, pois cada inserção gasta $O(\log n)$. Não é possível inseri-los em tempo constante.

Por que a C é a Correta?

Para ordenar n elementos por comparação, existem $n!$ permutações possíveis (folhas na árvore de decisão). Como cada comparação bifurca a árvore em no máximo 2 caminhos, a altura mínima h da árvore no pior caso é:

$$h \geq \lceil \log_2(n!) \rceil$$

Pela Aproximação de Stirling ($\ln(n!) \approx n \ln n - n$), temos que $\log_2(n!) = \Omega(n \log n)$.

Logo, nenhum algoritmo de comparação supera $\Omega(n \log n)$.

O Counting Sort e o Radix Sort quebram essa regra porque não comparam elementos entre si: eles usam a representação inteira do número para indexar um vetor na memória RAM em tempo $O(1)$, atingindo tempo linear sob condições favoráveis.

Questão 02: A Dependência Mandatória da Estabilidade (POSCOMP Adaptada)

Enunciado:

(POSCOMP adaptada) Um algoritmo de ordenação é classificado como estável se a ordem relativa dos itens com chaves de mesmo valor mantém-se inalterada após a execução. A respeito dessa propriedade, considere as afirmativas:

- O algoritmo Counting Sort padrão é estável se o vetor original de entrada for percorrido da direita para a esquerda (do final para o início) durante a escrita no vetor de saída.
- O algoritmo Radix Sort LSD depende obrigatoriamente da estabilidade do algoritmo de ordenação intermediário utilizado em cada passada para garantir a corretude global do vetor final.

III. O algoritmo Quick Sort clássico (com partição de Hoare ou Lomuto) é inerentemente estável e *in-place*.

Quais afirmativas estão corretas?

- **A)** Apenas I.
- **B)** Apenas II.
- **C)** Apenas I e II.
- **D)** Apenas II e III.
- **E)** I, II e III.

Análise das Alternativas Incorretas:

- **A, B e D estão incorretas** porque falham na identificação correta da combinação de proposições verdadeiras (omitem I ou II, ou aceitam a afirmativa III).
- **Por que a afirmativa III é FALSA?** O Quick Sort padrão realiza trocas (*swaps*) entre elementos distantes ao redor do pivô. Elementos de valores idênticos podem ser transpostos através do pivô, alterando a sua ordem relativa de entrada. Logo, o Quick Sort clássico é instável (embora seja *in-place*). Isso elimina imediatamente as alternativas D e E.

Por que a C é a Correta (I e II são verdadeiras)?

- **Afirmativa I (Verdadeira):** O Counting Sort acumula as contagens em C , onde $C[x]$ indica a última posição onde a chave x deve ser escrita. Ao ler o vetor original A de trás para frente, o último item lido com chave x é colocado exatamente na posição $C[x] - 1$. O acumulador é decrementado para $C[x] - 1$, garantindo que a ocorrência anterior da mesma chave seja posicionada imediatamente à esquerda. Assim, a ordem original é preservada perfeitamente.
- **Afirmativa II (Verdadeira):** O Radix Sort LSD ordena a partir do dígito menos significativo para o mais significativo. Suponha que já tenhamos ordenado as dezenas dos números 42 e 48 (resultado: 42, depois 48). Quando formos ordenar pela centena (dígito mais significativo), ambos têm a mesma centena 4. Se o algoritmo não for estável, 48 pode ser colocado antes de 42, destruindo a ordenação prévia das unidades. Portanto, a estabilidade é um requisito matemático do LSD.

Questão 03: Rastreio Passo a Passo do Counting Sort

Enunciado:

Considere a execução do Counting Sort sobre o vetor de entrada $A = [3, 1, 4, 1, 3]$ cujos elementos pertencem ao intervalo $[1, 4]$. Qual será o conteúdo do vetor de

frequências acumuladas C após a etapa de somas prefixadas e qual o primeiro elemento a ser gravado no vetor de saída B , considerando a implementação estável padrão?

- **A)** $C = [2, 0, 2, 1]$; o primeiro elemento gravado é 1 na posição 0.
- **B)** $C = [0, 2, 2, 4, 5]$; o primeiro elemento gravado é 3 na posição 3.
- **C)** $C = [1, 2, 4, 5]$; o primeiro elemento gravado é 4 na posição 4.
- **D)** $C = [0, 1, 2, 3, 4]$; o primeiro elemento gravado é 1 na posição 1.
- **E)** $C = [2, 2, 4, 5]$; o primeiro elemento gravado é 3 na posição 4.

Análise das Alternativas Incorretas:

- **Por que a A está incorreta?** O vetor $[2, 0, 2, 1]$ representa apenas as contagens simples (frequências isoladas dos números 1, 2, 3 e 4), mas a questão exige o vetor de frequências acumuladas ($C[i] = C[i] + C[i - 1]$).
- **Por que a C está incorreta?** O número de elementos menores ou iguais a 1 é 2 (existem dois números '1' na entrada). A opção C diz que $C[1] = 1$, o que ignoraria a repetição.
- **Por que a D está incorreta?** Apresenta uma progressão aritmética simples que não reflete a frequência dos dados em A .
- **Por que a E está incorreta?** Se o vetor começa no índice 1, $C[1] = 2$, $C[2] = 2$, $C[3] = 4$ e $C[4] = 5$. No entanto, ao gravar o elemento de chave 3, ele vai para o índice $C[3] - 1 = 4 - 1 = 3$ (em indexação base 0), e não na posição 4.

Por que a B é a Correta?

Vamos executar os passos formais do algoritmo sobre $A = [3, 1, 4, 1, 3]$ ($n = 5, k = 4$):

1. Vetor de Frequências Simples (tamanho $k + 1 = 5$, índices 0 a 4):

- $C[0] = 0$ (não há zeros)
- $C[1] = 2$ (ocorrências do número 1)
- $C[2] = 0$ (não há número 2)
- $C[3] = 2$ (ocorrências do número 3)
- $C[4] = 1$ (ocorrência do número 4)

2. Somas Prefixadas / Acumuladas ($C[i] = C[i] + C[i - 1]$):

- $C[0] = 0$
- $C[1] = 0 + 2 = 2$
- $C[2] = 2 + 0 = 2$
- $C[3] = 2 + 2 = 4$
- $C[4] = 4 + 1 = 5$

Logo: $C = [0, 2, 2, 4, 5]$.

3. Distribuição Estável (varrendo A de trás para frente):

- O último elemento de A é $A[4] = 3$.
- Consultamos $C[3]$, cujo valor é 4.
- A posição no vetor de saída B (base 0) é $C[3] - 1 = 4 - 1 = 3$.
- Gravamos o elemento 3 na posição 3 de B e decrementamos $C[3]$ para 3.

Questão 04: A Degeneração do Counting Sort por Amplitude (Concurso / POSCOMP)

Enunciado:

O Counting Sort possui complexidade assintótica de tempo dada por $\Theta(n + k)$, onde n é o número de elementos e k é a amplitude dos valores das chaves ($k = \max(A) - \min(A) + 1$). Em qual das seguintes situações o uso do Counting

Sort torna-se completamente inviável, sendo superado em tempo e memória por um algoritmo $O(n \log n)$ como o Merge Sort?

- **A)** Na ordenação de 10.000.000 de números inteiros uniformemente distribuídos entre 0 e 100.
- **B)** Na ordenação de um vetor de 100.000 inteiros que já se encontra em ordem inversa (decrescente).
- **C)** Na ordenação de um vetor com $n = 1.000$ inteiros cujas chaves assumem valores arbitrários entre 1 e $2^{31} - 1$.
- **D)** Na ordenação de 500.000 registros em que todas as chaves possuem exatamente o mesmo valor inteiro.
- **E)** Na ordenação de um vetor onde $k = 2n$.

Análise das Alternativas Incorretas:

- **Por que a A não é o pior caso?** Aqui temos $n = 10^7$ e $k = 100$. O termo dominante é n , logo $\Theta(n + k) = \Theta(n)$ (tempo linear). O Counting Sort é extremamente rápido nesse cenário e consome apenas alguns bytes para o vetor C .
- **Por que a B não é o pior caso?** O Counting Sort não realiza comparações; vetores ordenados, quase-ordenados ou em ordem inversa executam exatamente o mesmo número de passos de laço. A inversão não degenera o algoritmo.
- **Por que a D não é o pior caso?** Se todos os itens têm o mesmo valor, $k = 1$. O tempo será $\Theta(n + 1) = \Theta(n)$, que é o melhor caso possível.

- **Por que a E não é o pior caso?** Se $k = 2n$, então $\Theta(n + k) = \Theta(n + 2n) = \Theta(3n) = \Theta(n)$. O algoritmo continua perfeitamente linear e muito eficiente.

Por que a C é a Correta?

Quando $n = 1.000$ e $k \approx 2 \times 10^9$ (inteiro de 32 bits):

1. **Espaço:** O algoritmo precisa alocar um vetor C com 2^{31} posições de inteiros. Isso exigiria mais de 8 Gigabytes de memória RAM apenas para o vetor de contadores, para ordenar míseros 1.000 números!
2. **Tempo:** O primeiro laço precisa inicializar o vetor C com zeros e depois percorrê-lo para as somas acumuladas. Isso gastará $\approx 2 \times 10^9$ operações. Enquanto isso, o Merge Sort gastaria apenas:

3. $n \log_2 n = 1.000 \times \log_2(1.000) \approx 1.000 \times 10 = 10.000$ operações
4. O Counting Sort executaria 200.000 vezes mais operações que o Merge Sort e esgotaria a memória. Por isso ele é restrito a domínios onde $k = O(n)$.

7. Materiais de Apoio:

Materiais de Apoio e Referências Práticas

Videoaulas e Demonstrações Conceituais

- [Radix Sort Algorithm Introduction in 5 Minutes \(CS Dojo\)](#)
- [Construção e Análise de Algoritmos: Radix Sort \(Prof. Rudini Sampaio / UFC\)](#)

Bancos de Questões e Exames Oficiais

- [Repositório Awesome-POSCOMP \(GitHub\)](#)
- [Repositório ENADE Computação \(GitHub\)](#)

Outras fontes de explicações

- <https://www.geeksforgeeks.org/dsa/radix-sort/>
- <https://www.geeksforgeeks.org/dsa/counting-sort/>

8. Referências

- AHO, Alfred V.; HOPCROFT, John E.; ULLMAN, Jeffrey D. **Data Structures and Algorithms**. Boston: Addison-Wesley, 1983.
- CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Introduction to Algorithms**. 4. ed. Cambridge: MIT Press, 2022.
- MORIN, Pat. **Open Data Structures: An Introduction**. Edmonton: Athabasca University Press, 2013. Disponível em: <https://opendatastructures.org/>.